01 Blueprint tecnico e arquitetura

Documento consolidado | leitura visual + especificacao operacional | rev. ambientes 23/10/2026

Visao executiva

Produto de estudo estilo YPT, com organizacao do estudo e nucleo social gratuito, revisoes inteligentes no Pro e monetizacao por assinatura. Fase 0 aprova fluxos antes do desenvolvimento funcional.

Arquitetura vigente

FRONTEND • React + Vite / PWA • hospedagem Vercel Pro

↓

API • Node.js + Express • hospedagem Render • MVC + Service (regras de negocio)

↓

DADOS E SERVICOS • Supabase: projeto DEV (Free) e PROD (Pro no lancamento)

↓

PAGAMENTOS • Mercado Pago, por integracao controlada na API

Ambientes e fluxo de trabalho aprovados

Um repositório GitHub para o produto inteiro. Todos clonam o mesmo projeto e desenvolvem em branches com Pull Requests; o GitHub guarda o código, não executa a API.

Componente	DESENVOLVIMENTO / TESTES	PRODUÇÃO (no lançamento)
Código / colaboração	GitHub único: backend/, frontend/, docs/; clones locais, Issues e PRs	Mesmo repositório; somente commits revisados chegam à main
Backend Node/Express	API executada localmente nos computadores de Gabriel e Samuel	1 Web Service no Render; sem segunda instância para homologação inicial
Banco, Auth e Realtime	Supabase plantel-dev, Free, com dados fictícios e migrations autorizadas	Supabase plantel-prod separado, Pro antes da entrada em produção
Frontend React/Vite	Execução local + ambiente Preview do mesmo projeto Vercel	Ambiente Production do mesmo projeto Vercel

Fluxo: issue → branch local → pull request → revisão/testes → merge. Migrations: arquivo versionado → plantel-dev → teste e revisão → plantel-prod (somente com autorização).

Cobrança Supabase: uma mesma conta pode administrar os dois projetos. Quando o produto entrar no Pro, projetos Free e Pro precisam ficar em organizações diferentes para manter planos distintos; validar custos e separação antes do upgrade.

Fronteiras de responsabilidade

Componente	Faz	Nao faz
Frontend	Telas, acessibilidade, experiencia responsiva e estado visual	Decidir Pro, permissao ou pontuacao confiavel
API Express	Regras, autorizacao, tempo de sessao, ranking, pagamentos	Armazenar senhas ou expor segredos ao navegador
Supabase Auth	Credenciais, login, recuperacao e JWT	Substituir verificacao de acesso do backend

Componente	Faz	Nao faz
PostgreSQL Supabase	Dados oficiais e integridade relacional	Executar regras sensiveis somente no cliente
Supabase Realtime	Broadcast, Presence e canais privados autorizados	Comprovar tempo real de estudo
Render	Executar API Node.js; logs e deploy	Hospedar banco de dados do Plantel

Escopo comprometido com o lancamento

Nucleo	Entrega V1
Identidade	Cadastro com nome, email, senha, apelido e data_nascimento; email confirmado; privacidade e onboarding
Estudo	Hoje, metas, timer bloco/livre/Pomodoro, sessoes cronometradas e manuais, historico
Social gratuito	Grupos, convite, mesa de presenca, mural sem texto livre, reacoes e ranking de grupo
Revisoes Pro	1/7/30; dificuldade, carga diaria, folgas, atrasos e retomada
Comercial	Teste Pro de 14 dias, assinatura mensal/anual, pagamento e reconciliacao
Operacao	Resumo domingo, notificacoes, tres hierarquias administrativas, auditoria e LGPD

Nao comprometer para o lancamento

- Duelos individuais, sala do dia e cards de compartilhamento: somente possivel 1.0.x, fora das Issues ativas do lancamento.
- IA semanal, simulados, rankings publicos, moedas, loja, desafios ampliados e chat livre: fora da V1.
- MongoDB Atlas: somente arquitetura futura de chat; nao contratar, criar colecoes ou implementar agora.
- Indicações/recompensas e meta coletiva constam da base, mas seus criterios de concessao precisam de validacao de produto/antiabuso; nao usar telefone.

Convencoes e implementacao

Padrao de nomenclatura do codigo — frontend e backend

Regra transversal da V1: todo codigo novo deve usar nomes de dominio em portugues brasileiro sem acentos. As palavras da linguagem e os nomes tecnicos padronizados permanecem em ingles. O Codex deve aplicar essa regra nos dois lados do produto.

Escopo	Regra obrigatoria
Identificadores de negocio	Portugues do Brasil, sem acentos nem cedilha, com nomes claros: usuarioId, dadosValidados, iniciarSessao(), calcularTempoEstudado(), enviarMensagem(). Aplicar tambem a parametros, variaveis locais, constantes, objetos e suas propriedades novas, funcoes, componentes React, hooks proprios, arquivos e modulos de dominio.
Convencoes de escrita	JavaScript/JSX: camelCase para variaveis, parametros, funcoes, propriedades e hooks proprios; PascalCase para componentes React e classes quando necessarias; UPPER_SNAKE_CASE para constantes globais. SQL: snake_case em nomes de tabelas e colunas de dominio, respeitando o modelo de dados aprovado.
Elementos que NAO se traduzem	Palavras reservadas e APIs do JavaScript/Node/React/SQL mantem sua sintaxe original: async, await, function, return, const, let, try, catch, require, module.exports, useState, useEffect, map, filter, SELECT, JOIN etc. Nomes obrigatorios de bibliotecas, frameworks, protocolos e convencoes de camadas tambem: controller, service, model, routes, middlewares, shared, components, hooks, services, config.
Contratos e compatibilidade	Nao renomear endpoints, nomes de campos JSON, tabelas, colunas, variaveis de ambiente ou interfaces JA APROVADOS apenas para traduzir. Respeitar documentos 02 e 03 e codigo

Escopo	Regra obrigatoria
	existente; qualquer mudanca de contrato requer revisao conjunta e migracao planejada.
Codigo limpo e exemplos	Preferir nomes completos e significativos, funcoes pequenas, responsabilidade unica e comentarios em portugues quando necessarios. Evitar siglas obscuras e atalhos sem contexto. Exemplos, novos testes, mocks e subprompts devem seguir a mesma politica.

O arquivo AGENTS.md, versionado na raiz do repositório, deve reforçar esta regra em toda Issue, junto com a verificacao de Git/PRs/Projects. A criacao do arquivo exige aprovacao do responsavel e nao autoriza outras alteracoes.

- Backend: modules/<dominio> com routes/controller/service/model; camadas tecnicas em ingles, dominios em portugues ASCII.
- SQL PostgreSQL usando pg + Pool, placeholders, migrations via node-pg-migrate; sem ORM.
- JWT validado no backend. usuario_id deve vir da identidade autenticada, nao do request body.
- Manter observabilidade, testes, rate limit, idempotencia, logs seguros e boas praticas LGPD.

Ordem de entrega

F0: inventario e prototipo responsivo → revisao tecnica → aprovacao de produto

↓

S0: fundacao tecnica e ambientes (preparo pode ocorrer em paralelo)

↓

S1–S4: funcionalidades da V1 por dependencia

↓

Beta: E2E, seguranca, legal e homologacao

↓

Estabilizacao: performance, correcoes e aceite final

Hospedagem e capacidade

Render executará a API apenas em produção; na fase de desenvolvimento ela roda localmente. Dimensionar o Render por testes e simultaneidade, sem presumir capacidade. PostgreSQL permanece no Supabase em dois projetos isolados: dev (Free) e prod (Pro no lançamento). Vercel hospeda o frontend com Preview/Production no mesmo projeto. Nao inferir capacidade de usuarios apenas de CPU/RAM.